

Wykład 3

Deep Learning w chmurze

Klasyfikacja. Ocena jakości modelu klasyfikacyjnego.

Dr Urszula Grzybowska
Katedra Zastosowań Matematyki
p. 95 Budynek 34

Wykład 3

Zastosowanie sieci neuronowych do klasyfikacji.

Miary oceny jakości klasyfikacji

Wykłady, które polecam

MIT Introduction to Deep Learning | 6.S191



Alexander Amini

343 tys. subskrybentów



Stanford CS230: Deep Learning ...

Stanford Online - 1/9

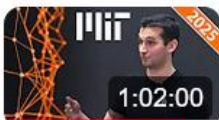


- ▶  1:00:17
Stanford CS230 | Autumn 2025 |
Lecture 1: Introduction to Deep...
Stanford Online
- 2  1:39:48
Stanford CS230 | Autumn 2025 |
Lecture 2: Supervised, Self-...
Stanford Online
- 3  1:07:05
Stanford CS230 | Autumn 2025 |
Lecture 3: Full Cycle of a DL...
Stanford Online
- 4  1:47:17
Stanford CS230 | Autumn 2025 |
Lecture 4: Adversarial...
Stanford Online
- 5  1:45:01
Stanford CS230 | Autumn 2025 |
Lecture 5: Deep Reinforcement...
Stanford Online



Andrew Ng

https://www.youtube.com/watch?v=_NLHFoVNIbg&list=PLoROMvodv4rNRRGdS0rBbXOUGA0wjdh1X

- ▶  1:09:26
MIT Introduction to Deep Learning
| 6.S191
Alexander Amini
- 2  1:01:34
MIT 6.S191: Recurrent Neural
Networks, Transformers, and...
Alexander Amini
- 3  1:01:04
MIT 6.S191: Convolutional Neural
Networks
Alexander Amini
- 4  48:57
MIT 6.S191: Deep Generative
Modeling
Alexander Amini
- 5  1:02:00
MIT 6.S191: Reinforcement
Learning
Alexander Amini

Przykład



Kosaciec, inaczej irys

Zestaw danych kwiatu irysa Fishera to wielowymiarowy zestaw danych wykorzystywany i rozstawiony przez brytyjskiego statystyka i biologa **Ronalda Fishera** w jego artykule z 1936 r. Wykorzystanie wielu pomiarów w problemach taksonomicznych jako przykład liniowej analizy dyskryminacyjnej. Czasami nazywany jest zestawem danych Andersona, ponieważ Edgar Anderson zebrał dane w celu ilościowego określenia zmienności morfologicznej kwiatów irysów trzech pokrewnych gatunków.

Zestaw danych składa się z 50 próbek z każdego z trzech gatunków Iris (**Iris setosa**, **Iris virginica** i **Iris versicolor**). W każdej próbce zmierzono cztery cechy: długość i szerokość działek i płatków w centymetrach. Na podstawie kombinacji tych czterech cech, Fisher opracował liniowy model dyskryminacyjny w celu rozróżnienia każdego gatunku.

Dane

sepal length	sepal width	petal length	petal width	
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5	3.6	1.4	0.2	Iris-setosa
5.4	3.9	1.7	0.4	Iris-setosa
4.6	3.4	1.4	0.3	Iris-setosa
5	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa

sepal length	długość działki
sepal width	szerokość działki
petal length	długość płatka
petal width	szerokość płatka

Iris setosa, Iris virginica i Iris versicolor

Przygotowanie danych

Wykorzystamy tylko 2 zmienne objaśniające, 2 cechy: szerokość płatków i szerokość działki.

```
df = pd.read_csv(r'/content/Iris3.csv', sep=';')  
print(df.shape)
```

(150, 3)

```
df
```

	x1	x2	y
0	0.2	3.5	Iris-setosa
1	0.2	3.0	Iris-setosa
2	0.2	3.2	Iris-setosa
3	0.2	3.1	Iris-setosa
4	0.2	3.6	Iris-setosa
...	...	...	...
145	2.3	3.0	Iris-virginica

Dane są w pliku csv

```
df.dtypes
```

0

```
x1    float64
```

```
x2    float64
```

```
y      object
```

x1	x2	y
0.2	3.5	Iris-setosa
0.2	3	Iris-setosa
0.2	3.2	Iris-setosa
0.2	3.1	Iris-setosa
0.2	3.6	Iris-setosa
0.4	3.9	Iris-setosa
0.3	3.4	Iris-setosa
0.2	3.4	Iris-setosa
0.2	2.9	Iris-setosa

x1 szerokość płatków
x2 szerokość działki

Przygotowanie danych

Zmienna celu ma 3 kategorie.

```
df['y'].unique()
```

```
array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

```
y1=pd.Categorical(df['y']).rename_categories({'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2})
```

```
y1
```

```
[0, 0, 0, 0, 0, ..., 2, 2, 2, 2, 2]
```

```
Length: 150
```

```
Categories (3, int64): [0, 1, 2]
```

Przygotowanie danych

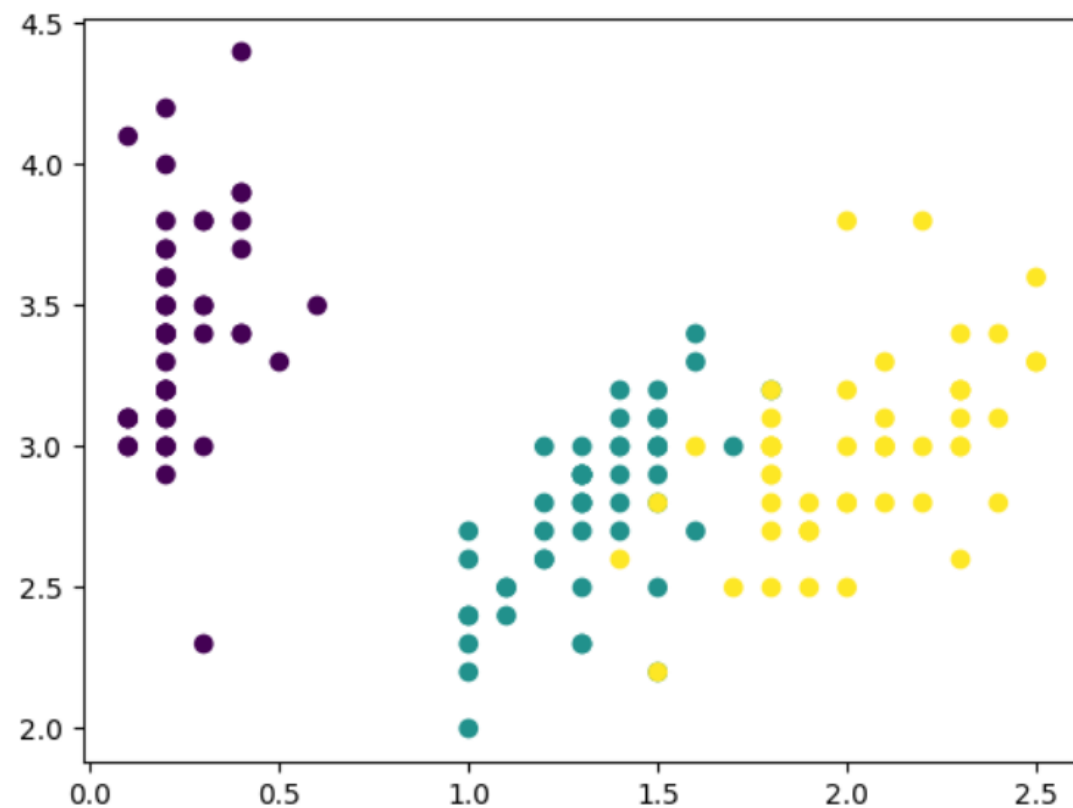
```
X=torch.from_numpy(df1.values).type(torch.float)
y=torch.tensor(y1).type(torch.LongTensor)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
len(X_train), len(X_test), len(y_train), len(y_test)
```

```
(120, 30, 120, 30)
```

```
import matplotlib.pyplot as plt
plt.scatter(x=df1["x1"], y=df1["x2"], c=y1)
plt.show()
```



Budowa modelu Sposób 1

Definiujemy nową klasę o nazwie **IrisModel**, która dziedziczy z **nn.Module**.
nn.Module jest klasą bazową dla wszystkich modułów sieci neuronowych w PyTorch

```
import torch
from torch import nn
```

```
class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
        self.relu =nn.ReLU()
    def forward(self, x):
        return self.layer_2(self.relu(self.layer_1(x)))
```

Model IRIS1

Budowa modelu Sposób 1

```
import torch
from torch import nn
```

```
class IrisModel(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
```

```
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
```

```
        self.relu =nn.ReLU()
```

```
    def forward(self, x):
```

```
        return self.layer_2(self.relu(self.layer_1(x)))
```

Konstruktor klasy IrisModel. Jest on wykonywany podczas tworzenia instancji tej klasy.

Model IRIS1

Budowa modelu Sposób 1

```
import torch
from torch import nn
```

Konstruktor klasy IrisModel. Jest on wykonywany podczas tworzenia instancji tej klasy.

```
class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
        self.relu =nn.ReLU()
    def forward(self, x):
        return self.layer_2(self.relu(self.layer_1(x)))
```

Wywołuje konstruktor klasy nadrzędnej (nn.Module), zapewniając prawidłową inicjalizację komponentów sieci neuronowej

Model IRIS1

Budowa modelu Sposób 1

```
import torch
from torch import nn
```

```
class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
        self.relu =nn.ReLU()
    def forward(self, x):
        return self.layer_2(self.relu(self.layer_1(x)))
```

Warstwy sieci, pierwszą
warstwę sieci neuronowej
jest warstwa liniowa



Model IRIS1

Budowa modelu

```
import torch
from torch import nn
```

```
class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
        self.relu =nn.ReLU()
    def forward(self, x):
        return self.layer_2(self.relu(self.layer_1(x)))
```

definiuje propagację w przód sieci neuronowej, sposób, w jaki dane przepływają przez warstwy podczas uczenia lub predykcji

Model IRIS1

Budowa modelu

```
import torch
from torch import nn
```

```
class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1=nn.Linear(in_features=input_features, out_features=hidden_units)
        self.layer_2=nn.Linear(in_features=hidden_units, out_features=output_features)
        self.relu =nn.ReLU()
    def forward(self, x):
        return self.layer_2(self.relu(self.layer_1(x)))
```

Opisuje sekwencję operacji w przejściu do przodu. Wejście x jest najpierw przekazywane przez warstwę_1, następnie funkcja aktywacji ReLU (self.relu) jest stosowana do wyjścia warstwy_1, a na koniec wynik jest przekazywany przez warstwę_2 w celu uzyskania ostatecznego wyjścia

Model IRIS1

Budowa modelu

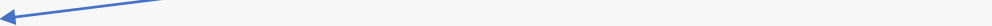
```
input_features=2  
hidden_units=2  
output_features=3
```

```
Iris_model1=IrisModel()  
Iris_model1
```

```
IrisModel(  
    (layer_1): Linear(in_features=2, out_features=2, bias=True)  
    (layer_2): Linear(in_features=2, out_features=3, bias=True)  
    (relu): ReLU()  
)
```

Budowa modelu Sposób2

```
NUM_Features=2  
NUM_CLASSES=3
```

```
class IrisModel(nn.Module):  
    def __init__(self, input_features, output_features, hidden_units=2):  
        super().__init__()   
        self.linear_layer_stack=nn.Sequential(  
            nn.Linear(in_features=input_features, out_features=hidden_units),  
            nn.ReLU(),  
            nn.Linear(in_features=hidden_units, out_features=output_features)  
        )  
    def forward(self,x):  
        return self.linear_layer_stack(x)
```

inicjalizuje klasę nadrzędną (nn.Module), zapewniając, że wszystkie niezbędne komponenty sieci neuronowej są poprawnie skonfigurowane

```
Iris_model1=IrisModel(input_features=NUM_Features, output_features=NUM_CLASSES, hidden_units=2)  
Iris_model1
```

```
IrisModel(  
  (linear_layer_stack): Sequential(  
    (0): Linear(in_features=2, out_features=2, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=2, out_features=3, bias=True)  
  )  
)
```

Model IRIS2

Budowa modelu Sposób 2

```
NUM_Features=2  
NUM_CLASSES=3
```

```
class IrisModel(nn.Module):  
    def __init__(self, input_features, output_features, hidden_units=2):  
        super().__init__()  
        self.linear_layer_stack=nn.Sequential(  
            nn.Linear(in_features=input_features, out_features=hidden_units),  
            nn.ReLU(),  
            nn.Linear(in_features=hidden_units, out_features=output_features)  
        )  
    def forward(self,x):  
        return self.linear_layer_stack(x)
```

warstwy modelu są
definiowane i układane
razem przy użyciu
nn.Sequential

```
Iris_model1=IrisModel(input_features=NUM_Features, output_features=NUM_CLASSES, hidden_units=2)  
Iris_model1
```

```
IrisModel(  
  (linear_layer_stack): Sequential(  
    (0): Linear(in_features=2, out_features=2, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=2, out_features=3, bias=True)  
  )  
)
```

Model IRIS2

Budowa modelu Sposób 2

```
NUM_Features=2  
NUM_CLASSES=3
```

```
class IrisModel(nn.Module):  
    def __init__(self, input_features, output_features, hidden_units=2):  
        super().__init__()  
        self.linear_layer_stack=nn.Sequential(  
            nn.Linear(in_features=input_features, out_features=hidden_units),  
            nn.ReLU(),  
            nn.Linear(in_features=hidden_units, out_features=output_features)  
        )  
    def forward(self,x):  
        return self.linear_layer_stack(x)
```

```
Iris_model1=IrisModel(input_features=NUM_Features, output_features=NUM_CLASSES, hidden_units=2)  
Iris_model1
```

```
IrisModel(  
  (linear_layer_stack): Sequential(  
    (0): Linear(in_features=2, out_features=2, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=2, out_features=3, bias=True)  
  )  
)
```

Mamy dwie liniowe (w pełni połączone) warstwy. Pierwsza pobiera input_features jako dane wejściowe i wyprowadza wartości hidden_units. Druga przyjmuje hidden_units jako dane wejściowe i wyprowadza wartości output_features.

nn.ReLU(): Jest to funkcja aktywacji stosowana po pierwszej warstwie liniowej. ReLU (Rectified Linear Unit) wprowadza nieliniowość do modelu, co ma kluczowe znaczenie dla uczenia się złożonych wzorców.

Model IRIS2

Budowa modelu Sposób 2

```
NUM_Features=2
```

```
NUM_CLASSES=3
```

```
class IrisModel(nn.Module):
    def __init__(self, input_features, output_features, hidden_units=2):
        super().__init__()
        self.linear_layer_stack=nn.Sequential(
            nn.Linear(in_features=input_features, out_features=hidden_units),
            nn.ReLU(),
            nn.Linear(in_features=hidden_units, out_features=output_features)
        )
    def forward(self,x):
        return self.linear_layer_stack(x)
```

Definiuje sposób przepływu danych przez model podczas przejścia do przodu (przewidywanie). Pobiera ona dane wejściowe (x) i przepuszcza je przez warstwę linear_layer_stack zdefiniowaną w konstruktorze. Dane wyjściowe linear_layer_stack są następnie zwracane jako prognoza modelu.

```
Iris_model1=IrisModel(input_features=NUM_Features, output_features=NUM_CLASSES, hidden_units=2)
Iris_model1
```

```
IrisModel(
  (linear_layer_stack): Sequential(
    (0): Linear(in_features=2, out_features=2, bias=True)
    (1): ReLU()
    (2): Linear(in_features=2, out_features=3, bias=True)
  )
)
```

Budowa modelu Sposób 2

```
NUM_Features=2  
NUM_CLASSES=3
```

```
class IrisModel(nn.Module):  
    def __init__(self, input_features, output_features, hidden_units=2):  
        super().__init__()  
        self.linear_layer_stack=nn.Sequential(  
            nn.Linear(in_features=input_features, out_features=hidden_units),  
            nn.ReLU(),  
            nn.Linear(in_features=hidden_units, out_features=output_features)  
        )  
    def forward(self,x):  
        return self.linear_layer_stack(x)
```

Tworzy instancję klasy IrisModel i przypisuje ją do zmiennej Iris_model1.
Konkretny model tworzony według przepisu IrisModel

```
Iris_model1=IrisModel(input_features=NUM_Features, output_features=NUM_CLASSES, hidden_units=2)  
Iris_model1
```

```
IrisModel(  
  (linear_layer_stack): Sequential(  
    (0): Linear(in_features=2, out_features=2, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=2, out_features=3, bias=True)  
  )  
)
```

Model IRIS2

Budowa modelu Sposób 3

```
class IrisModel1(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1 = nn.Linear(in_features=2, out_features=2)
        self.layer_2 = nn.Linear(in_features=2, out_features=3) # extra layer
        self.relu = nn.ReLU() # <- dodanie ReLU jako funkcji aktywacji

    def forward(self, x):
        x = self.layer_1(x)
        x = self.relu(x)
        x = self.layer_2(x)
        return x
...
# inna możliwość
return self.layer_3(self.relu((self.layer_2(self.relu(self.layer_1(x))))))
...

Iris_model1 = IrisModel1()
print(Iris_model1)

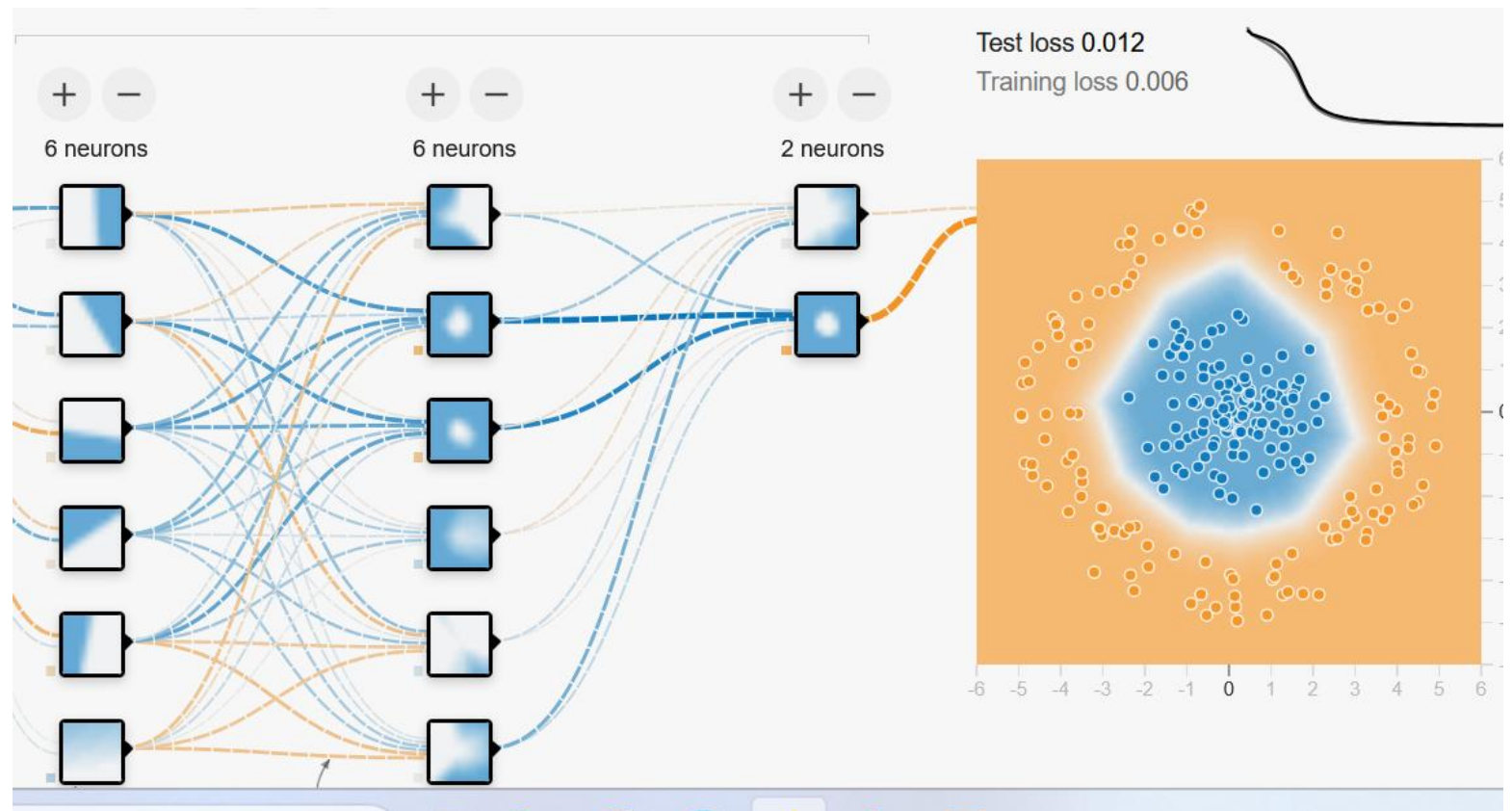
def count_parameters(Iris_model1):
    return sum(p.numel() for p in Iris_model1.parameters() if p.requires_grad)
print(f'The model has {count_parameters(Iris_model1):,} trainable parameters')
```

```
IrisModel1(
  (layer_1): Linear(in_features=2, out_features=2, bias=True)
  (layer_2): Linear(in_features=2, out_features=3, bias=True)
  (relu): ReLU()
)
The model has 15 trainable parameters
```

Model IRIS3

Tensorflow Playground

- Link do animacji: <https://playground.tensorflow.org>



Wybór funkcji straty i optymalizatora

```
loss_fn=nn.CrossEntropyLoss()  
optimizer=torch.optim.SGD(Iris_model1.parameters(), lr=0.1)
```

Wykład 2 Obliczenia_lossW2

```
def accuracy_fn(y_true, y_pred):  
    correct = torch.eq(y_true, y_pred).sum().item() # torch.eq() calculates where two tensors are equal  
    acc = (correct / len(y_pred)) * 100  
    return acc
```

Ocena modelu

Można też korzystać z gotowych funkcji:

```
from sklearn.metrics import classification_report  
print(classification_report(y_preds, y_test))
```

Accuracy

```
CLASS ignite.metrics.Accuracy(output_transform=<function  
Accuracy.<lambda>>, is_multilabel=False, device=device(type='cpu'),  
skip_unrolling=False) [SOURCE]
```

Calculates the accuracy for binary, multiclass and multilabel data.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

```
!pip install torchmetrics
```

```
from torchmetrics import Accuracy
```

```
from torchmetrics.functional import accuracy
```

Uczenie modelu

```
# Wybierz ziarno
torch.manual_seed(42)

# Ustal liczbę epok
epochs = 100

for epoch in range(epochs):
    ### Uczenie
    Iris_model1.train()

    # 1. Propagacja w przód
    y_logits = Iris_model1(X_train) # wyrzuca logity
    y_pred = torch.softmax(y_logits, dim=1).argmax(dim=1) # przejście z logitów -> prawdopodobieństwa -> przewidywane etykiety
    # print(y_logits)
    # 2. Strata i dokładność
    loss = loss_fn(y_logits, y_train)
    acc = accuracy_fn(y_true=y_train,
                      y_pred=y_pred)

    # 3. Wyzarowanie gradientów
    optimizer.zero_grad()

    # 4. Propadacja wsteczna
    loss.backward()

    # 5. Optymalizator
    optimizer.step()
```


Testowanie/Predykcja

```
### Testowanie
Iris_model1.eval()
with torch.inference_mode():
    # 1. Propagacja w przód
    test_logits = Iris_model1(X_test)
    test_pred = torch.softmax(test_logits, dim=1).argmax(dim=1)
    # 2. Strata i dokładność
    test_loss = loss_fn(test_logits, y_test)
    test_acc = accuracy_fn(y_true=y_test,
                           y_pred=test_pred)
```

Predykcja przy pomocy modelu

```
Iris_model1.eval()  
with torch.inference_mode():  
    y_logits = Iris_model1(X_test)  
  
# Podgląd 10 pierwszych predykcji  
y_logits[:10]
```

```
tensor([[ 0.0666,  1.8209,  1.0483],  
        [ 3.8362,  0.8258, -2.8992],  
        [-3.4786,  3.0757,  4.9061],  
        [-0.7397,  2.1689,  1.9543],  
        [-0.5402,  2.0504,  1.7153],  
        [ 3.1172,  0.9255, -2.1873],  
        [-0.1329,  1.9394,  1.2872],  
        [-2.9591,  3.0945,  4.4332],  
        [-1.4670,  2.1425,  2.6164],  
        [-0.0373,  1.8171,  1.1429]])
```

Predykcja

```
# Zamiana logitów w prawdopodobieństwa
y_pred_probs = torch.softmax(y_logits, dim=1)

# Zamiana prawdopodobieństw na etykiety
y_preds = y_pred_probs.argmax(dim=1)

# Porównanie pierwszych 10 preds i test labels
print(f"Przewidywania: {y_preds[:10]}\nLabels: {y_test[:10]}")
print(f"Dokładność: {accuracy_fn(y_true=y_test, y_pred=y_preds)}%")
```

```
Przewidywania: tensor([1, 0, 2, 1, 1, 0, 1, 2, 2, 1])
Labels: tensor([1, 0, 2, 1, 1, 0, 1, 2, 1, 1])
Dokładność: 93.33333333333333%
```

Przykład W3_Iris_Siec.xls

Macierz pomyłek

Macierz pomyłek została wynaleziona w 1904 roku przez Karla Pearsona. Używał on terminu Contingency Table.

		Predicted condition	
		Predicted Positive (PP)	Predicted Negative (PN)
Actual condition	Total population = P + N		
	Positive (P) ^[a]	True positive (TP), hit ^[b]	False negative (FN), type II error, miss, underestimation ^[c]
	Negative (N) ^[d]	False positive (FP), type I error, false alarm, overestimation ^[e]	True negative (TN), correct rejection ^[f]

Macierz	Wynik klasyfikacji	Model		
		Pozytywny	Negatywny	Razem
Obserwacje empiryczne	Pozytywny	TP	FN	TP+FN=P
	Negatywny	FP	TN	FP+TN=N
	Razem	TP+FP	FN+TN	P+N
				Trafność/dokładność klasyfikatora (TP+TN)/(P+N)

Miary oceny jakości klasyfikacji

Czułość (Recall) $\frac{TP}{TP+FN}$

Precyzja klasyfikatora

$$\frac{TP}{TP + FP}$$

Specyficzność

$$\frac{TN}{TN + FP} = 1 - \frac{FP}{TN + FP}$$

Oszacowanie prawdopodobieństwa błędnej klasyfikacji (misclassification rate MR)

$$MR = \frac{FP + FN}{P + N}$$

Macierz	Wynik klasyfikacji	Model		
		Pozytywny	Negatywny	Razem
Obserwacje empiryczne	Pozytywny	TP	FN	TP+FN=P
	Negatywny	FP	TN	FP+TN=N
	Razem	TP+FP	FN+TN	P+N
				Trafność/dokładność klasyfikatora (TP+TN)/(P+N)

$$f1 = \frac{2}{\frac{1}{precyzja} + \frac{1}{czułość}}$$

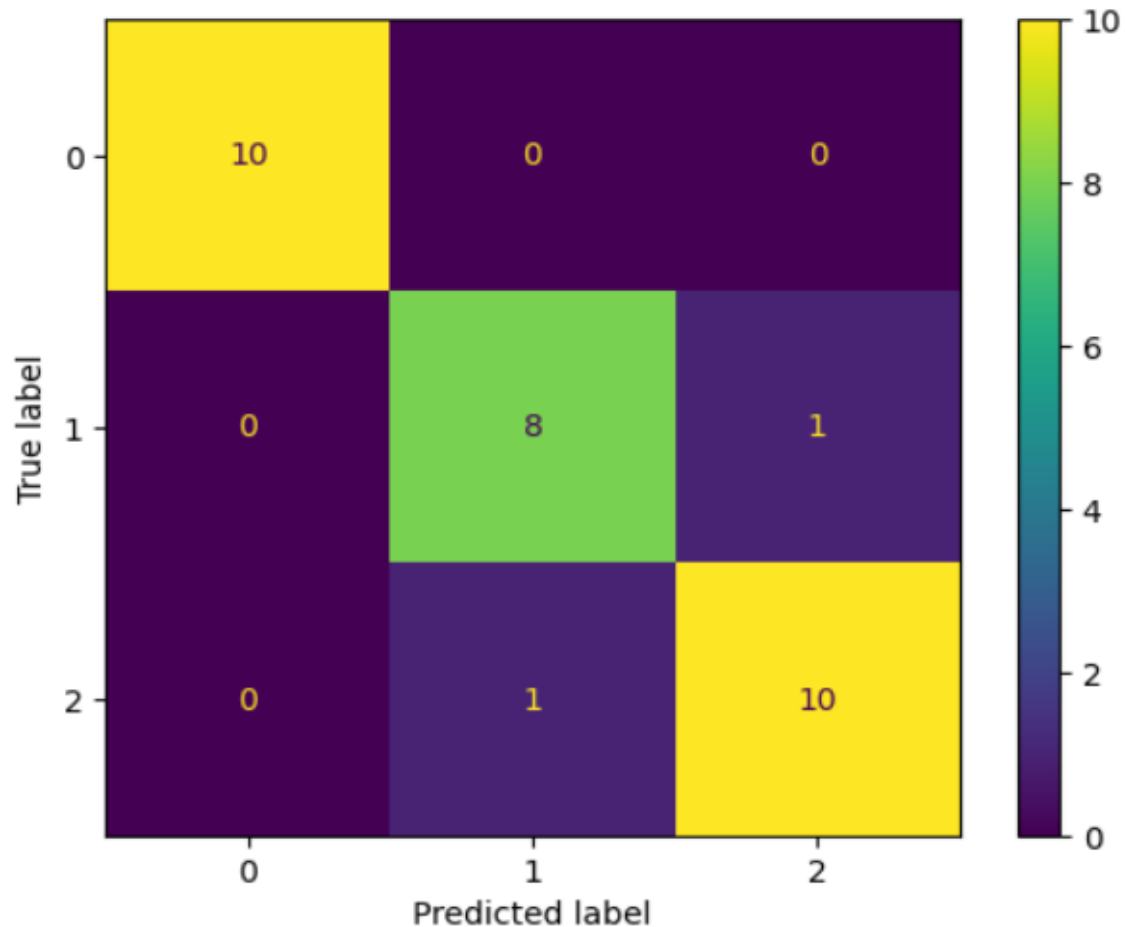
Im większy tym lepiej

$$f1 = \frac{TP}{TP + \frac{(FP + FN)}{2}}$$

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, ConfusionMatrixDisplay
```

```
# Create the confusion matrix  
cm = confusion_matrix(y_test, y_preds)
```

```
ConfusionMatrixDisplay(confusion_matrix=cm).plot();
```



```
from sklearn.metrics import classification_report  
print(classification_report(y_preds, y_test))
```

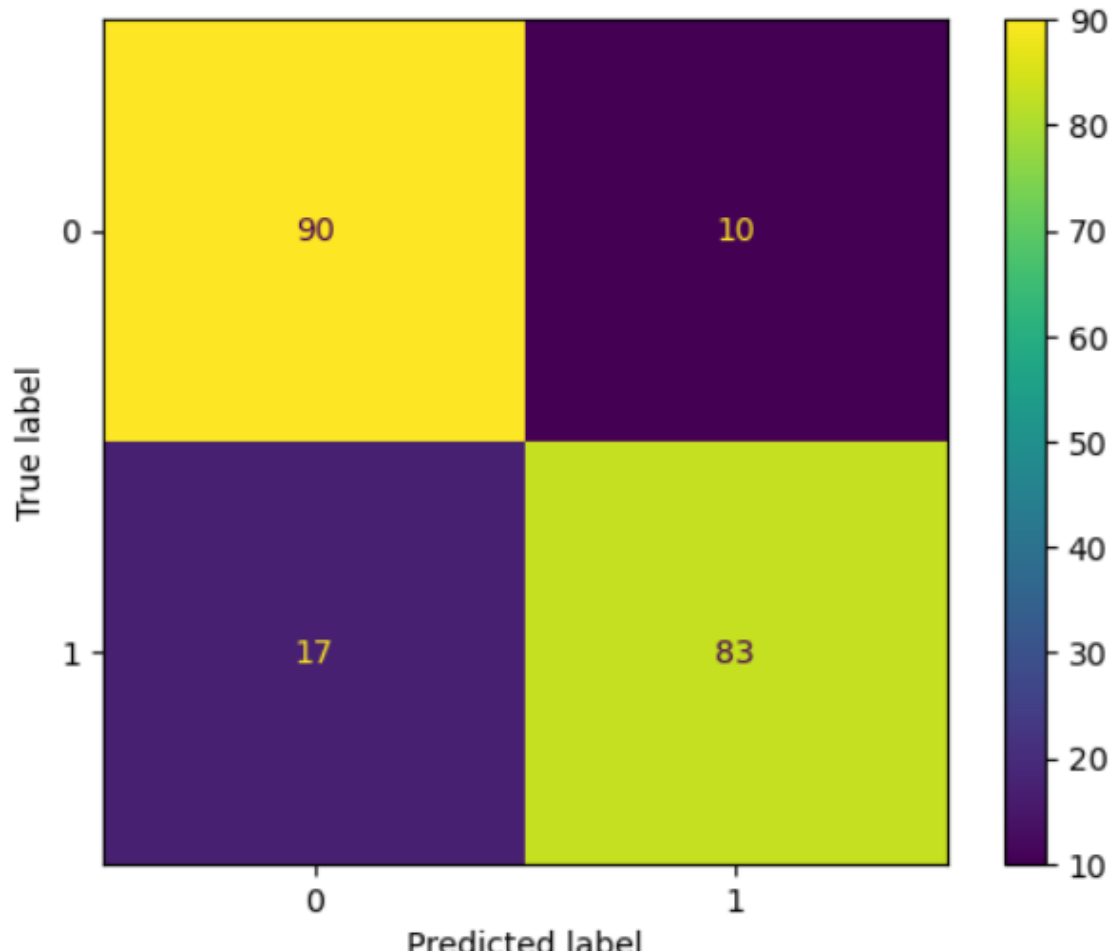
	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.89	0.89	0.89	9
2	0.91	0.91	0.91	11
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Miary oceny klasyfikatora

Tu robimy jeden vs pozostałe. Czyli wybieramy jedną klasę i sumujemy wartości nad i pod przekątną oraz na przekątnej dla pozostałych klas.

Przykład z ćwiczeń nr 3

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, ConfusionMatrixDisplay
# Create the confusion matrix
cm = confusion_matrix(y_test, y_preds)
ConfusionMatrixDisplay(confusion_matrix=cm).plot();
```

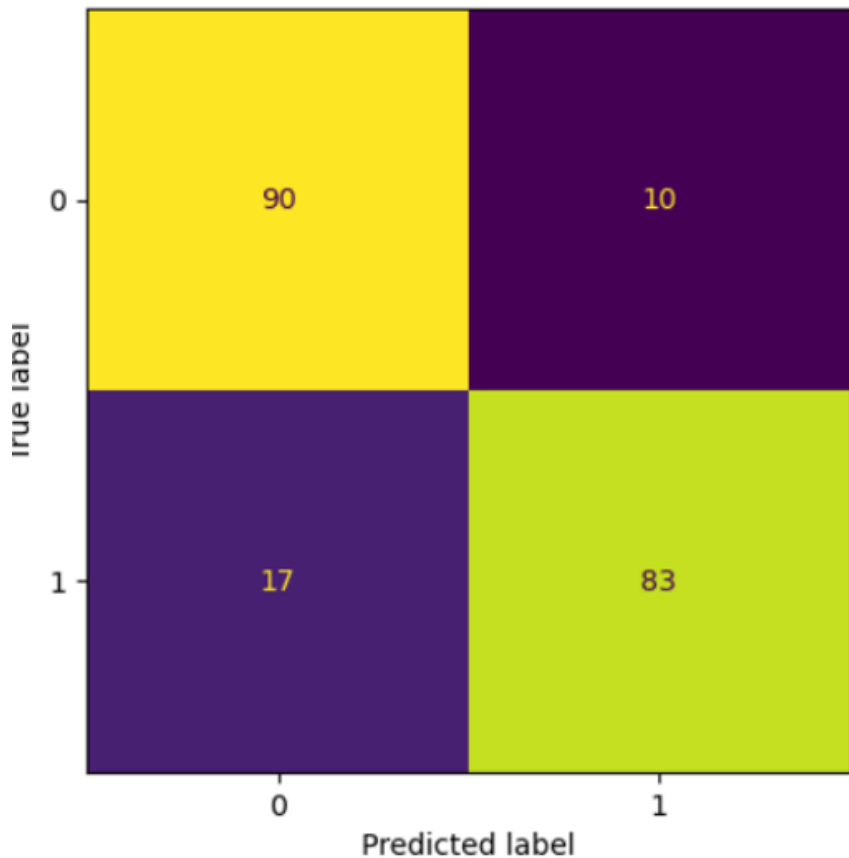


```
from sklearn.metrics import classification_report
print(classification_report(y_preds, y_test))
```

	precision	recall	f1-score	support
0.0	0.90	0.84	0.87	107
1.0	0.83	0.89	0.86	93
accuracy			0.86	200
macro avg	0.86	0.87	0.86	200
weighted avg	0.87	0.86	0.87	200

$$accuracy = \frac{TP + TN}{P + N}$$

Wzory



Czułość (Recall) $\frac{TP}{TP+FN} = \frac{83}{93} = 0.89$

Precyzja klasyfikatora

$$\frac{TP}{TP + FP} = \frac{83}{100}$$

Specyficzność

$$\frac{TN}{TN + FP} = 1 - \frac{FP}{TN + FP}$$

$$accuracy = \frac{TP+TN}{P+N} = \frac{173}{200} = 0.865$$

	precision	recall	f1-score	support
0.0	0.90	0.84	0.87	107
1.0	0.83	0.89	0.86	93
accuracy			0.86	200
macro avg	0.86	0.87	0.86	200
weighted avg	0.87	0.86	0.87	200

Krzywa ROC

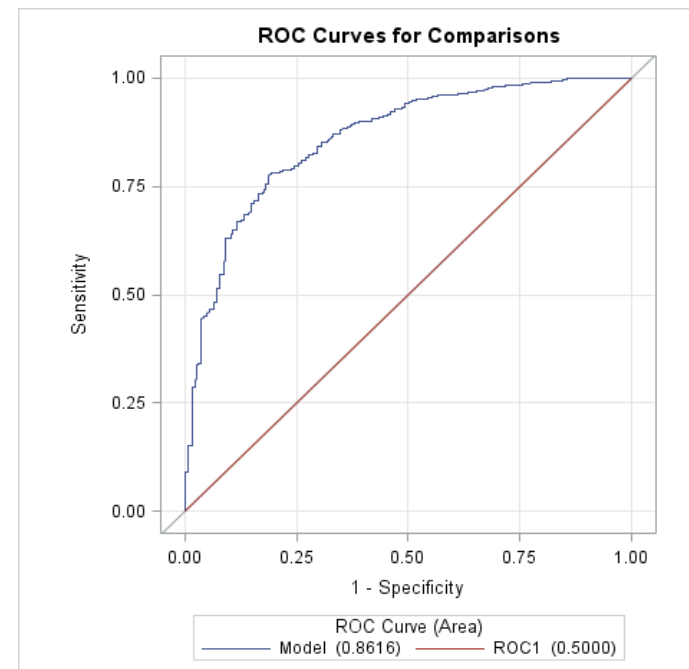
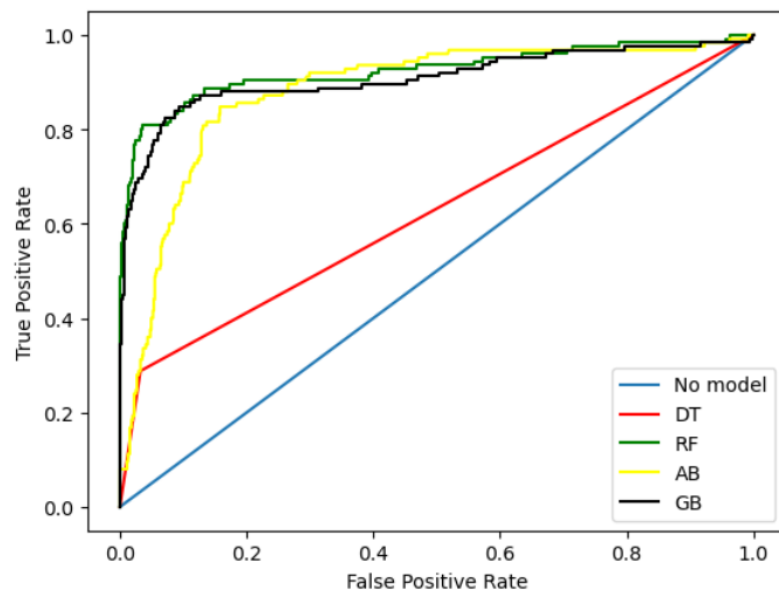
Krzywa ROC została po raz pierwszy opracowana przez inżynierów elektryków i inżynierów radarowych podczas II wojny światowej do wykrywania obiektów wroga na polach bitew, w 1941 roku, co doprowadziło do jej nazwy („charakterystyka operacyjna odbiornika”). Wkrótce została ona wprowadzona do psychologii w celu uwzględnienia percepcyjnego wykrywania bodźców.

Analiza ROC jest stosowana w medycynie, radiologii, biometrii i jest coraz częściej wykorzystywana w uczeniu maszynowym i badaniach eksploracji danych.

Krzywa ROC

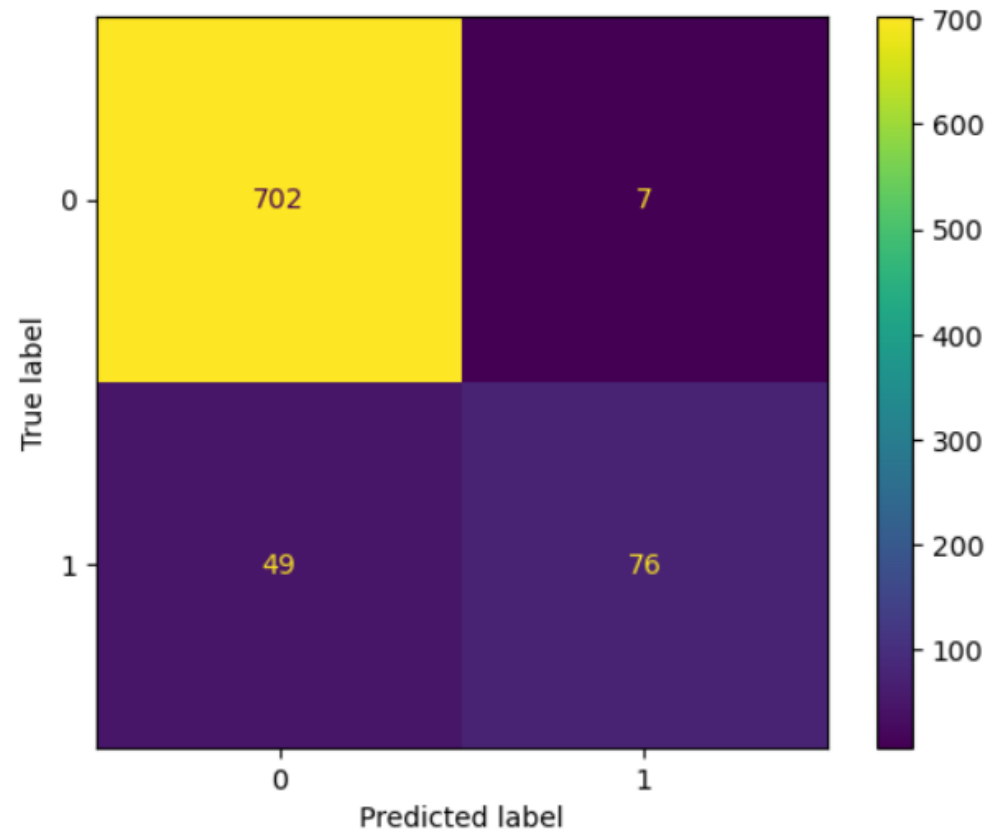
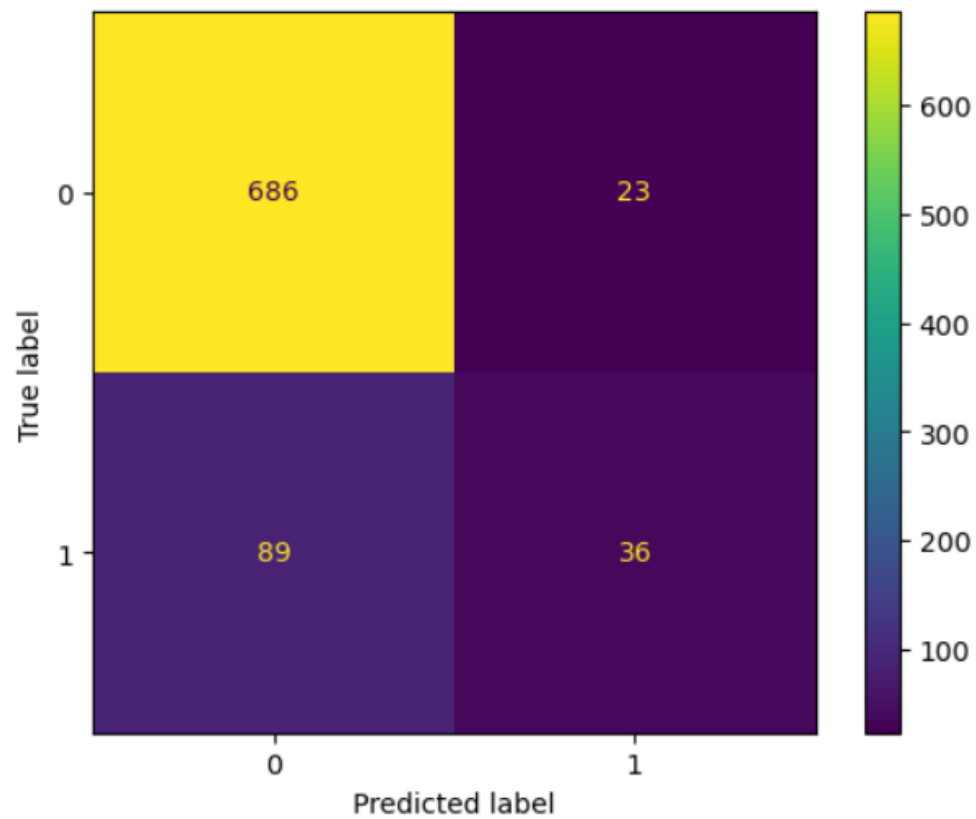
Krzywa ROC to miejsce geometryczne punktów o współrzędnych (1-specyficzność, czułość), czyli punktów o współrzędnych $(\frac{FP}{TN+FP}, \frac{TP}{TP+FN})$

No model: ROC AUC=0.500
DT: ROC AUC=0.628
RF: ROC AUC=0.924
AB: ROC AUC=0.880
GB: ROC AUC=0.908



Przykład dla danych Churn (dane telekomunikacyjne dotyczące abonentów telefonii komórkowej)

Dane niezbalansowane



Accuracy: 0.8657074340527577

AUC -area under the ROC curve

No model: ROC AUC=0.500
DT: ROC AUC=0.628
RF: ROC AUC=0.924
AB: ROC AUC=0.880
GB: ROC AUC=0.908

Dobłą miarą jest AUC. Im miara wyższa, tym lepiej.

Przykład w Excelu

No model: ROC AUC=0.500
DT: ROC AUC=0.628
RF: ROC AUC=0.924
AB: ROC AUC=0.880
GB: ROC AUC=0.908

